

Advanced Pandas

Pandas Advanced Techniques for Data Engineers

Pandas Dataframes Concatenation

Simple Concatenation- `pd.concat`

- Used to combine DataFrames along a particular axis (rows or columns).
- Syntax:

```
pd.concat([df1, df2])
```

- Default behavior keeps the original index.
- Useful when datasets have the same columns but are split across multiple files.

Ignoring the index

- Resets the index in the concatenated result.
- Useful when original indices are not meaningful.
- Example:

```
pd.concat([df1, df2], ignore_index=True)
```

Adding MultiIndex Keys

- Creates a hierarchical index to differentiate concatenated DataFrames.
- Helpful in distinguishing datasets from different sources.
- Example:

```
pd.concat([df1, df2], keys=['x', 'y'])
```

Concatenation with Joins

- `Pd.concat()` supports different types of joins:
 - inner (intersection)
 - outer (union, default)
- Ensures merging only relevant data based on index alignment.
- Example:

```
pd.concat([df1, df2], join='inner')
```

The Append Method

- Similar to `pd.concat()`, but deprecated in recent pandas versions.
- Use `pd.concat([df1, df2], ignore_index=True)` instead.
- Historically useful for quick appending of rows.

Joining Dataframes

The Join Categories

- **One-to-One Joins**

- Merging datasets with unique keys on both sides.
- Similar to SQL's INNER JOIN.

- **Many-to-One Joins**

- When one DataFrame contains duplicate key values.
- Common in lookup table operations.

- **Many-to-Many Joins**

- When both DataFrames contain duplicate key values.
- Used in scenarios like product-customer relationships.

```
pd.merge(df1, df2)
```

Specification of the Merge Key

- The *on* Keyword
 - Specifies the common column(s) to merge on.
- Example:

```
df1.merge(df2, on='key')
```

Specification of the Merge Key

- The *left_on* and *right_on* Keywords:
 - Used when column names differ in DataFrames.
 - Allows merging disparate datasets.
- Example:

```
df1.merge(df2, left_on='id1', right_on='id2')
```

Specification of the Merge Key

- The *left_index* and *right_index* Keywords:
 - Merging using index values instead of columns.
- Example:

```
df1.merge(df2, left_index=True, right_index=True)
```

Overlapping Column Names

- Use the *suffixes* Keyword
- Prevents column name conflicts.
- Ensures clarity when merging datasets with similar column names.
- Example:

```
df1.merge(df2, on='key', suffixes=('_left', '_right'))
```

Aggregation and Grouping

Simple Aggregation Functions

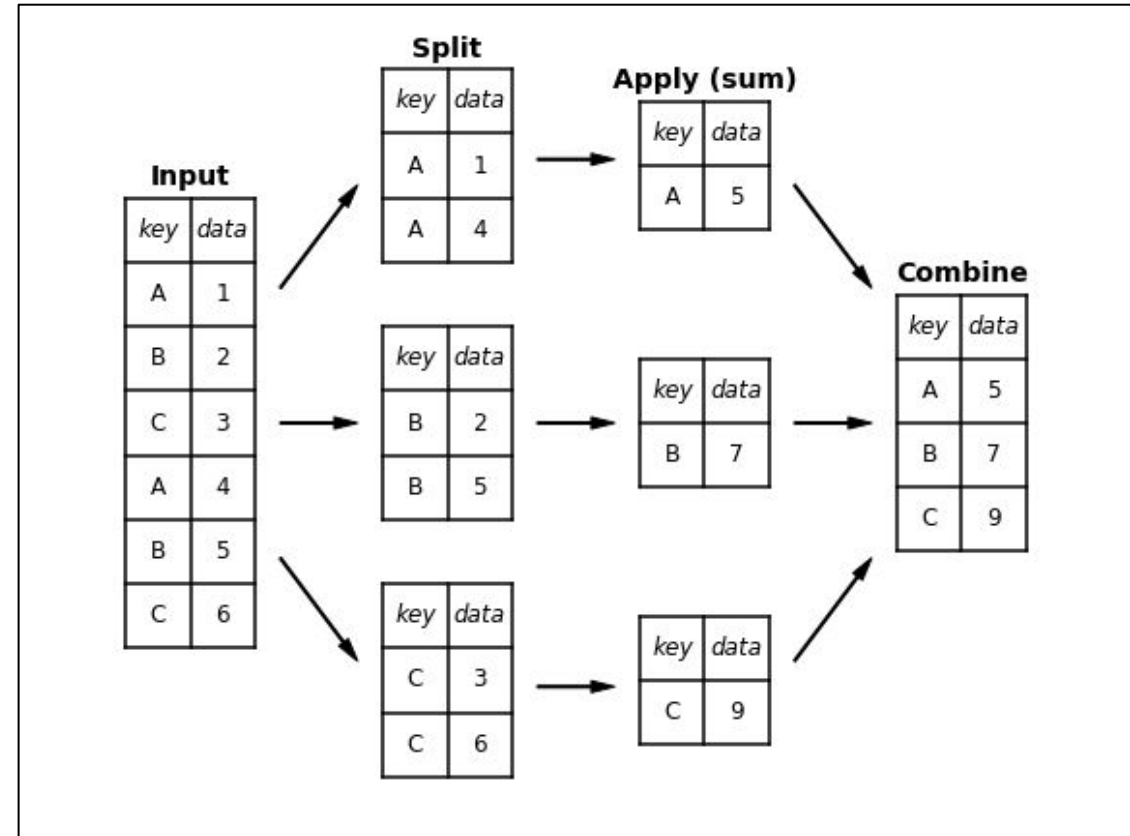
- Aggregation functions: *sum()*, *mean()*, *min()*, *max()*.
- Example:

```
df['column'].sum()  
df['column'].mean()  
df['column'].min()  
df['column'].max()
```


Groupby: Split, Apply, Combine

- Groups data on a specific filter, applies an aggregation and combines results to a new table.
- Example:

```
df.groupby('category')['value'].mean()
```



The *GroupBy* Object:

- **Column Indexing:**

- The *GroupBy* object supports column indexing like the *DataFrame* and returns a modified *GroupBy* object.
- This option allows you to select a specific group of the original data to apply your calculations on.

- **Iteration Over Groups:**

- Allows you to pick each of the returned groups as a series.

- **Dispatching:**

- And then you can transform it back to your original table using *unstack*.

```
for name, group in df.groupby('category'):  
    print(name)  
    print(group)
```

Aggregate, Filter, Transform, Apply

- There are more additional functions available for grouping other than the main four aggregate functions.
- The GroupBy objects have *aggregate*, *filter*, *transform*, and *apply* methods.
- They efficiently implement a variety of useful operations before combining the grouped data.

The Aggregate Method

- Performs multiple aggregations allowing more flexibility and additional options for data calculations.
- Can take a string, a list or a function of data calculation steps.
- Example:

```
df.groupby('key').aggregate(['min', np.median, max])  
[20]
```

The Filter Method

- Remove groups based on conditions allowing you to drop data based on the group properties.
- Example:
- Keeping all groups in which the standard deviation is larger than some critical value.

```
def filter_func(x):  
    return x['data2'].std() > 4  
  
df.groupby('key').filter(filter_func)
```

The Transform Method

- Modify data while keeping its original shape.
- Example:
- Center the data by subtracting the group-wise mean.

```
def center(x):  
    return x - x.mean()  
df.groupby('key').transform(center)  
[23]
```

The Apply Method

- Apply custom functions to groups.
- Example:
- Normalizes the first column by the sum of the second.

```
def norm_by_data2(x):  
    # x is a DataFrame of group values  
    x['data1'] /= x['data2'].sum()  
    return x  
  
df.groupby('key').apply(norm_by_data2)  
[24]
```

Pivot Tables

Pivot Table Syntax

- Basic Syntax for Pivot Tables

```
df.pivot_table(values='fare', index=['sex', 'class'],  
columns='embark_town', aggfunc='mean')
```

- Multilevel Pivot Tables

```
df.pivot_table(values='fare', index=['sex', 'class'],  
columns='embark_town', aggfunc='mean')
```


Pivot Table Full Call

```
DataFrame.pivot_table(data, values=None, index=None,  
                        columns=None, aggfunc='mean',  
                        fill_value=None, margins=False,  
                        dropna=True, margins_name='All',  
                        observed=False, sort=True)
```

Working with Time Series in Pandas

Dates and Times in Python

- Using Python's *datetime* and *dateutil*.
- Built-in modules for handling date/time data in python.

```
from dateutil import parser
date = parser.parse("4th of July, 2021")
```

```
from datetime import datetime
datetime(year=2021, month=7, day=4)
```

- NumPy's *datetime64*
- Efficient for date/time storage and computation.

```
import numpy as np
date = np.array('2021-07-04', dtype=np.datetime64)
```

Dates and Times in Pandas

- Combines best features from *datetime* and *numpy*.
- Builds upon all the other tools to provide a `Timestamp` object, which combines the ease of use of `datetime` and `dateutil` with the efficient storage and vectorized interface of `numpy.datetime64`.

```
import pandas as pd  
date = pd.to_datetime("4th of July, 2021")
```

Indexing *DataFrames* by Time

- Is useful when you begin to index data by timestamps.
- For example,
 - You can construct a *Series* object that has time-indexed data:

```
index = pd.DatetimeIndex(['2020-07-04', '2020-08-04',  
                          '2021-07-04', '2021-08-04'])  
data = pd.Series([0, 1, 2, 3], index=index)
```

- And then access (index) it as in normal types:

```
data['2020-07-04':'2021-07-04']
```

Data Structures

- *Data structures for time series data:*
 - *pd.Timestamp*: A single timestamp and a replacement for Python's native *datetime*, but it's based on the more efficient *numpy.datetime64* data type.
 - *pd.DatetimeIndex*: Index of timestamps.
 - *pd.Period*: A time period (e.g., month, year) . It encodes a fixed-frequency interval based on *numpy.datetime64*. The associated index structure is *PeriodIndex*.
 - *Pd.Timedelta*: A more efficient replacement for Python's native *datetime.timedelta* type and is based on *numpy.timedelta64*.

Regular Sequences: *pd.date_range*

- Is used for creating a date range.
- Example:

```
pd.date_range('2023-01-01', periods=10, freq='D')
```


Regular Sequences: *pd.period_range*

- Is used for creating a period range.
- Example:

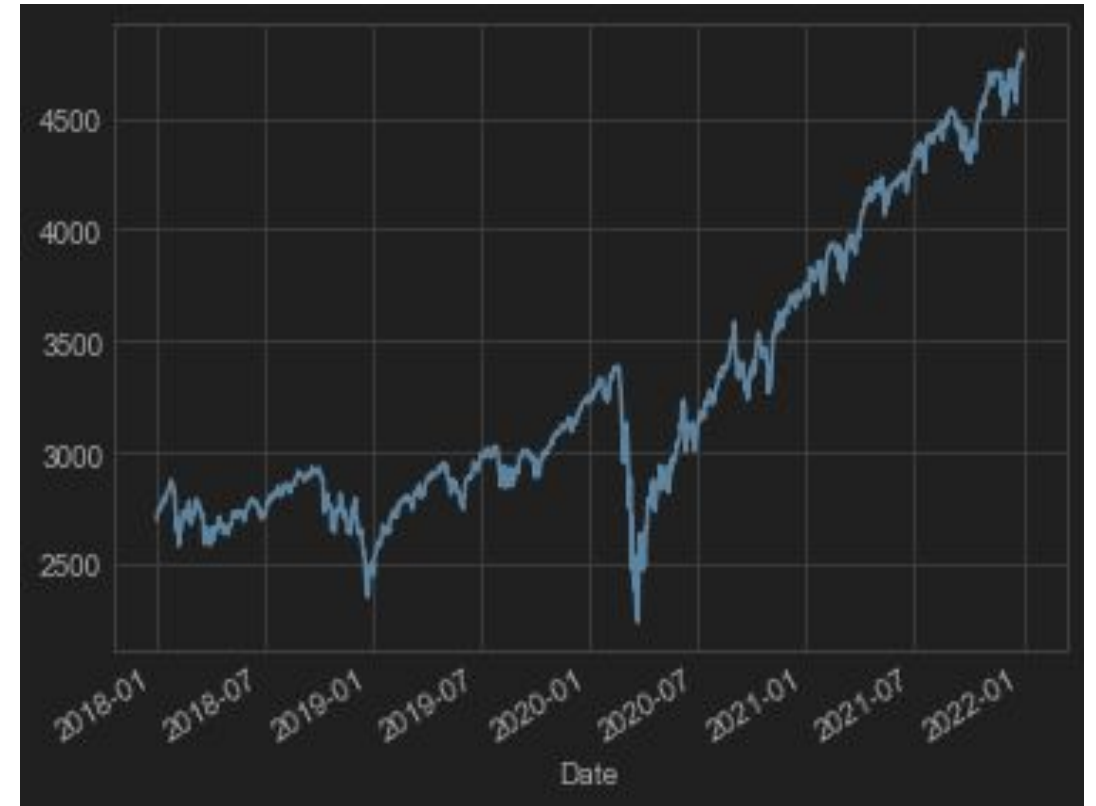
```
pd.period_range('2023-01', periods=10, freq='M')
```


Frequencies and Offsets

- *freq* parameter allows custom time intervals.
- Examples: 'D' (daily), 'M' (monthly), 'H' (hourly).

Code	Description	Code	Description
D	Calendar day	B	Business day
W	Weekly		
M	Month end	BM	Business month end
Q	Quarter end	BQ	Business quarter end
A	Year end	BA	Business year end
H	Hours	BH	Business hours
T	Minutes		
S	Seconds		
L	Milliseconds		
U	Microseconds		
N	Nanoseconds		

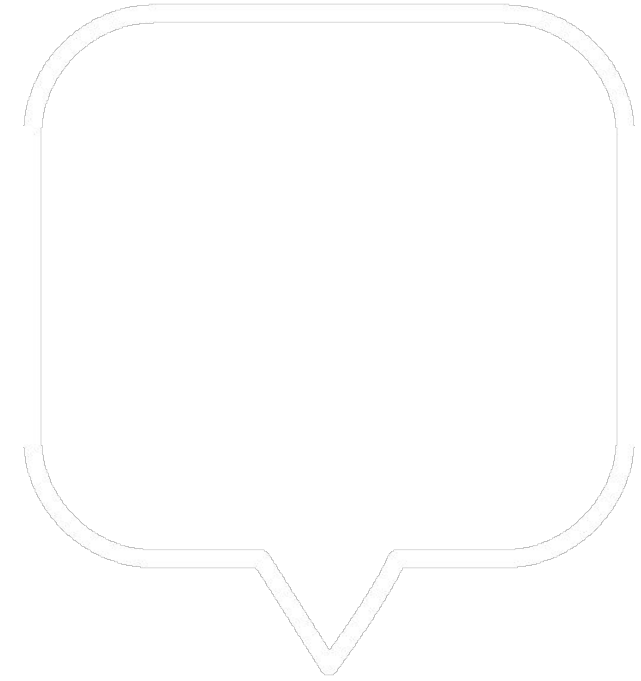
Practical Time Series Example (Stock Price)



Let's Code!



Any Questions?



Quiz Time!

Thanks